

INDENG 231 Course Project 2

Using LLMs to Build a Personal Knowledge Base: Food Delivery Dispatch Algorithms

Anna Chen

May 2026

1 What I Built

I built a lightweight LLM-powered knowledge base system focused on the topic of **food delivery platform dispatch algorithms** — the real-time decision-making pipelines that platforms like Meituan, Uber Eats, and DoorDash use to match orders to riders, optimize delivery routes, and pre-position idle couriers before demand spikes. I chose this topic because it sits at an interesting intersection of combinatorial optimization, machine learning, and real-time systems engineering, and because it connects directly to the themes in Project 1.

1.1 System Architecture

The system follows a three-stage pipeline inspired by Karpathy's workflow:

```
raw/ --> ingest.py (LLM) --> wiki/INDEX.md
                                wiki/concepts/  [[backlinked articles]]
                                wiki/algorithms/
                                --> qa.py (Q&A CLI)
```

Ingestion (ingest.py). The script reads each file in `raw/` and sends it to Claude (claude-opus-4-7) with a prompt asking it to produce a clean wiki article: structured with headings, a one-sentence **summary**: tag, and `[[concept]]` backlinks to related topics. The LLM output is saved directly into the appropriate `wiki/` subdirectory and a one-line entry is appended to `INDEX.md`. Because the prompt is the same for every source, adding new articles requires no changes to the pipeline — just drop a new file into `raw/` and run `python ingest.py`.

Q&A (qa.py). A simple CLI that answers natural-language questions against the wiki. When an `ANTHROPIC_API_KEY` is available, it selects the top-4 most relevant articles (by keyword overlap) and sends them together with the question to Claude for a synthesised answer. Without an API key, it falls back to displaying relevant article excerpts directly — making the system usable even without spending tokens on every query.

To try it, run `python qa.py` for interactive mode, or pass a question directly:

```
python qa.py "How does batch matching work and why is it better than greedy?"
python qa.py "How does reinforcement learning apply to dispatch?"
python qa.py "How do platforms pre-position idle riders before demand spikes?"
```

1.2 Knowledge Base Content

I seeded the `raw/` directory with four hand-written source documents summarising the academic and industry literature on dispatch:

1. **Dispatch Overview:** the three sub-problems (matching, routing, zone management), key metrics, and industry landscape.
2. **Matching Algorithms:** greedy, Hungarian algorithm, auction algorithm, batch matching, and integer programming formulations.
3. **Routing & Batching:** VRP variants, order batching trade-offs, 2-opt heuristics, dynamic re-routing, ETA prediction.

4. **ML & Reinforcement Learning:** demand forecasting, RL-based dispatch (DiDi KDD 2018, Meituan), GNNs for road graphs, fairness concerns.

The LLM compiled these into four interlinked wiki articles totalling around 1,800 words, automatically adding backlinks like `[[Bipartite Matching]]` and `[[ETA Prediction]]` that connect related concepts across articles.

2 What Worked

LLM-generated structure was immediately useful. The biggest surprise was how much the LLM improved the *organisation* of the raw material, not just its readability. My raw sources were dense and unstructured; Claude consistently identified which ideas deserved their own heading, surfaced comparisons I hadn't made explicit (e.g., online vs. offline matching, greedy vs. batch), and added cross-links that helped me see connections I'd missed.

Backlinks created a genuinely navigable graph. The `[[concept]]` convention is simple but powerful. After ingestion, ETA Prediction appeared as a backlinked concept in three separate articles, which made it immediately obvious that this is a central dependency of the whole dispatch system — not just a detail in the routing section. In Obsidian, these render as clickable graph links that make the knowledge base feel like a real wiki rather than a pile of notes.

The keyword-based Q&A fallback is more useful than expected. Without an API key, `qa.py` returns the top-2 relevant article excerpts. For targeted questions (“how does batch matching work?”), this was often enough to get a complete answer, because the wiki articles are already well-structured. The LLM-powered mode would synthesise a tighter, more direct answer — but the fallback mode proves the value of having a well-organised wiki independently of the Q&A layer.

3 What I Would Improve

Automatic source collection. Right now, source documents are written by hand. A more complete system would include a scraper or web-clipper that pulls arXiv abstracts, blog posts, or documentation pages directly into `raw/`, automatically converts them to clean markdown (e.g., using `trafilatura` or the Obsidian Web Clipper), and triggers `ingest.py` to update the wiki.

Incremental updates instead of full rewrites. The current `ingest.py` creates a new wiki article from scratch each time. A better approach would be to compare the new source against the existing wiki page and only update the sections that changed — reducing token usage and preserving manual annotations.

Richer Q&A with citation. The current Q&A prompt just asks Claude to answer from the wiki. A more robust version would ask it to cite specific article sections in its answer, making it easy to verify and trace claims back to sources. Retrieval-Augmented Generation (RAG) with embedding-based retrieval (instead of keyword overlap) would also improve relevance on longer, more nuanced questions.

LLM “health checks” on the wiki. Following Karpathy’s suggestion, I would add a `lint.py` script that periodically runs Claude over the entire wiki looking for inconsistencies (e.g., two articles giving conflicting definitions of the same concept), orphaned backlinks (a `[[concept]]` that has no corresponding article), and gaps worth filling (concepts referenced but never elaborated).

Visual output formats. Rendering outputs as Marp slides or matplotlib diagrams (e.g., a flowchart of the dispatch pipeline) and filing them back into the wiki would make the knowledge base much more useful for presentations and further study.